

GUI-Based Multi-Client Chat Application Using TCP

Computer Networks Laboratory – Assignment 6

Submitted by

Nitin Roy

Roll No: CSB24047

Tezpur University – ISEA Networking Internship

1. Objective

This assignment converts the terminal-based TCP chat application built in Assignment 5 into a graphical desktop application. The core networking logic — the multi-client TCP server, broadcast and private messaging, online-user tracking, and chat-history logging — was reused with minimal changes. The focus was on replacing the terminal client with a Tkinter GUI while keeping the socket layer functionally identical, and on introducing event-driven and multithreaded GUI programming so the interface stays responsive while messages are exchanged over the network.

2. GUI Design

The client interface was built using Tkinter and is organised into two screens that appear one after another in the same window: a login screen and a chat screen.

Login Window

The login window contains a title label, a username entry box, and a Connect button. Pressing Enter also triggers the connect action. If the username field is left empty, a validation warning dialog is shown and the connection is not attempted; only after a valid username is entered does the client open a socket to the server and switch to the chat screen.

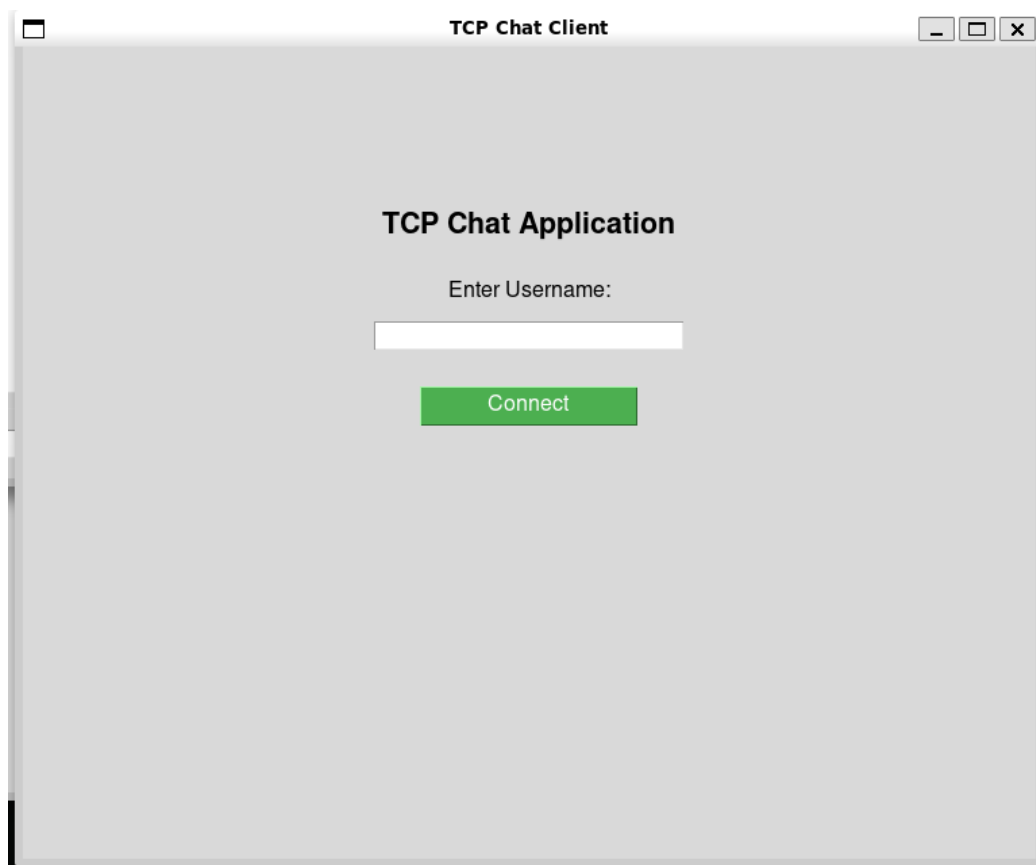


Fig. 2.1 – GUI login window with username entry and Connect button

Chat Window

The chat window is divided into a top bar showing the logged-in username and a Disconnect button, a middle area containing a scrollable message display alongside an online-users list, and a bottom bar with a message entry box and Send button. The message display auto-scrolls to the latest message, and pressing Enter in the input box sends the message just like clicking Send.

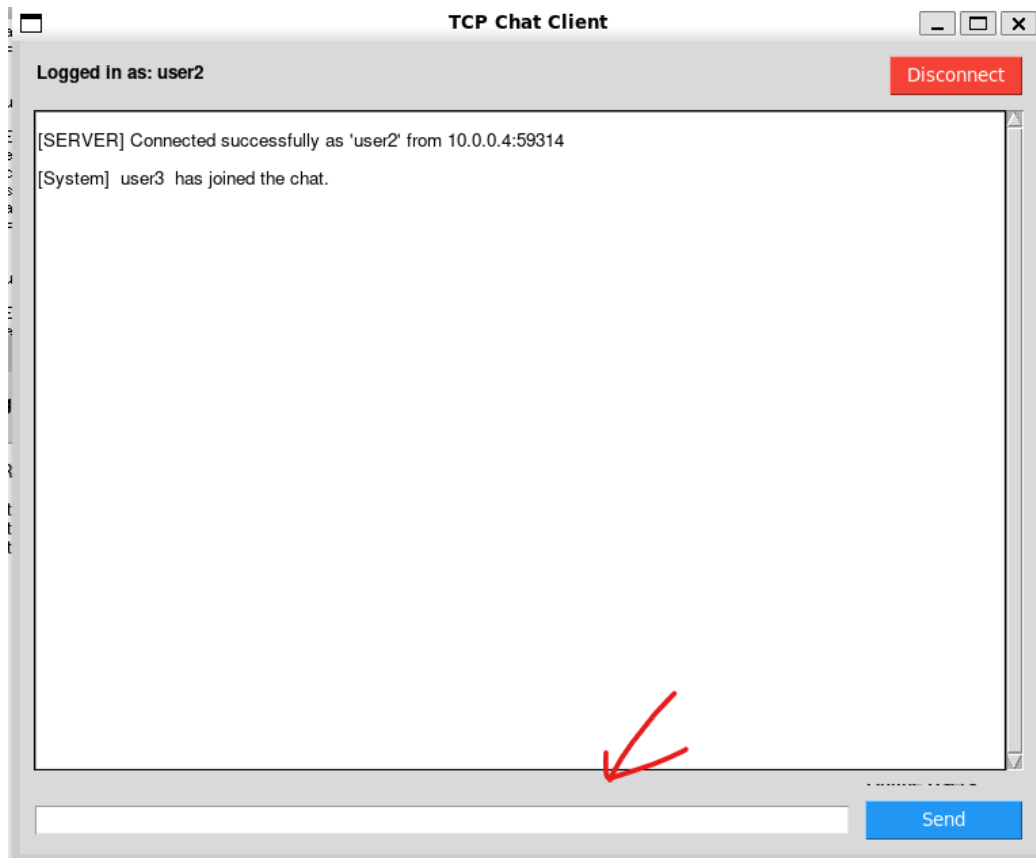


Fig. 2.2 – Chat window showing scrollable message area, online-users list, and input bar

3. System Architecture

The system follows a simple client–server architecture over TCP, deployed on a Mininet single-switch topology with one server host and four client hosts.

Topology: h1 = Chat Server | h2, h3, h4, h5 = Clients A–D | all hosts connected via switch s1

Mininet was started with a single-switch, five-host topology (`sudo mn --topo single,5`) and connectivity was verified before running the application.

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 s1
```

Fig. 3.1 – 'nodes' command listing available Mininet hosts

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0
```

Fig. 3.2 – 'net' command showing host-to-switch links

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
```

Fig. 3.3 – 'pingall' confirming 0% packet loss across all hosts

On the architecture side, the server (server.py) listens on port 5000 and spawns one worker thread per connected client to handle its socket I/O, while maintaining a shared dictionary of client sockets and states protected by a lock. Each GUI client (client_gui.py) opens one TCP connection to the server and runs a dedicated background thread purely for receiving data, so the Tkinter main loop is never blocked by network I/O.

4. Components Reused from Assignment 5

As required by the assignment, the server and the underlying networking logic were reused almost unchanged from Assignment 5. Only the client's presentation layer was replaced.

- TCP socket communication — the same connect/send/rcv sequence and port (5000) used in Assignment 5.
- Multi-client server — server.py accepts connections and spawns one thread per client, unchanged in structure.
- Broadcast messaging — broadcast_system_message() forwards a message to every online client except the sender.
- Private messaging — the /msg <username> <message> command routes a message to a single target client.
- Chat history logging — every broadcast, private message, and disconnect event is appended to chat_history.csv with a timestamp, sender, receiver, and message type.
- Client/session management — the clients dictionary tracking socket, IP, port, login time, and online/offline status, plus join/leave broadcast notifications.

Only the client-side interaction model changed: input() and print() calls from the Assignment 5 terminal client were replaced by Tkinter widgets and callback functions, and message reception was moved into its own thread instead of a blocking loop in the main thread.

5. GUI Design Decisions

- Two-frame flow: the login frame is destroyed and replaced by the chat frame after a successful connection, rather than using multiple windows, to keep the app in a single, simple window.
- ScrolledText for messages: a read-only, auto-scrolling ScrolledText widget is used for the chat log so users cannot accidentally edit history, while new lines always remain visible.
- Listbox for online users with selection-based private messaging: selecting a name in the Online Users list before sending automatically formats the message as a private /msg to that user, avoiding manual command typing.
- Background receiver thread: receive_messages() runs as a daemon thread started right after the chat window is built, so incoming JOIN/LEAVE/USERLIST/broadcast/private messages update the GUI without freezing the input box.
- Enter-key bindings on both the login and message entry fields, so the mouse is optional for common actions.
- Minimal, functional styling (colour-coded Connect/Disconnect/Send buttons) in line with the assignment's guidance to prioritise functionality over appearance.

6. Testing Results

The application was tested with the chat server running on h1 and four GUI clients running on h2–h5 inside the Mininet topology. The following behaviours were verified end-to-end:

- Successful login and connection for multiple simultaneous users (user1–user4).
- Broadcast messages from one client reaching all other connected clients.
- Private messages delivered only to the selected recipient, with a [Private from ...] tag.
- Automatic join notifications when a new user connects.
- Automatic leave notifications and list updates when a user disconnects.
- Clean disconnection using the Disconnect button, with the server logging the event to chat_history.csv.

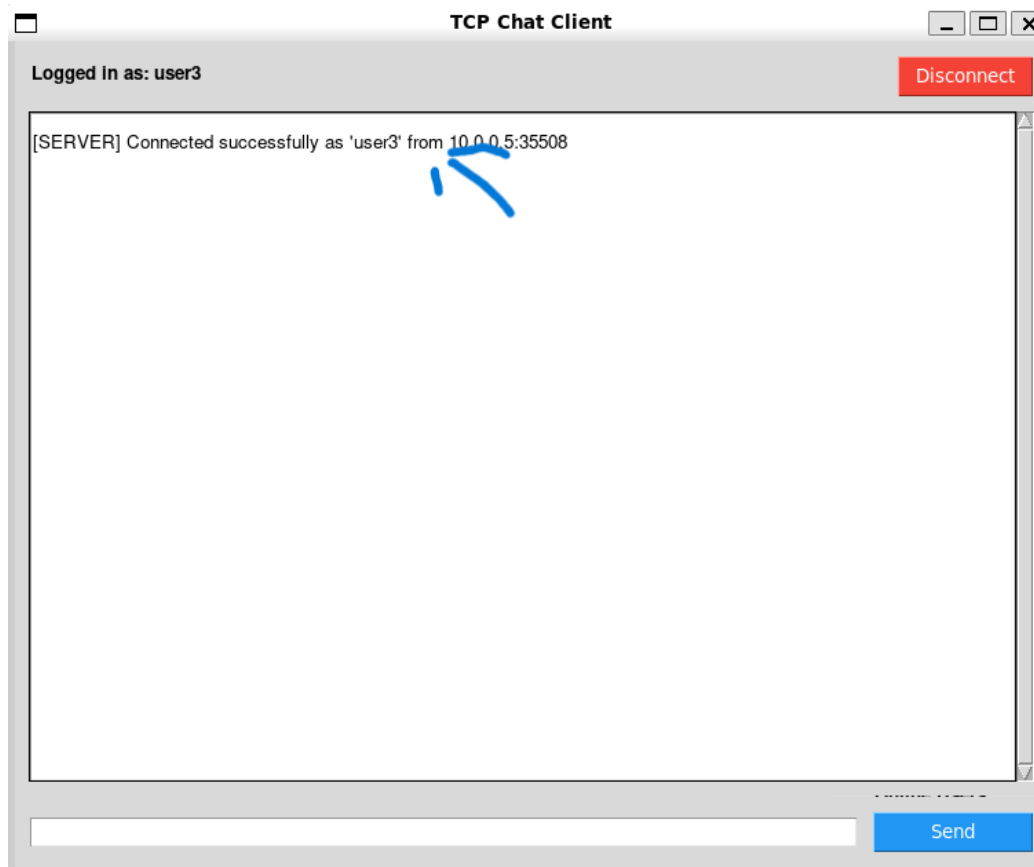


Fig. 6.1 – user3 successfully connected from 10.0.0.5:35508

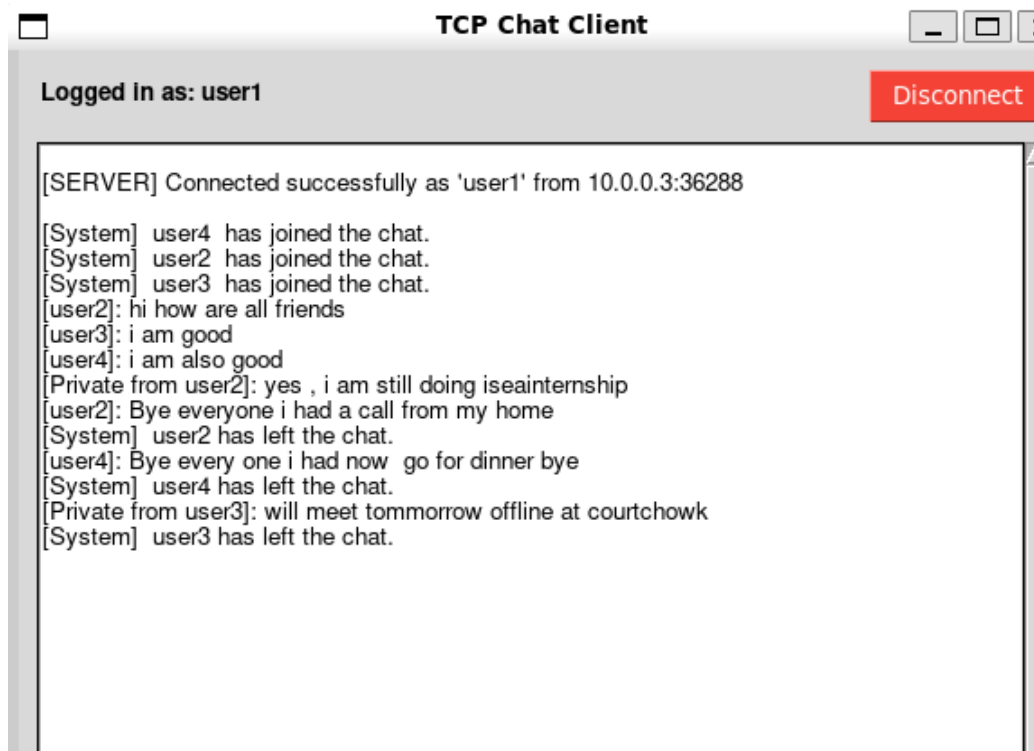


Fig. 6.2 – user1's session showing joins, broadcast chat, private messages, and leave notifications

The recorded conversation matches the server-side chat_history.csv log exactly — for example, user2's broadcast “hi how are all friends”, the private exchange between user1 and user2 about the internship, and the leave events for user2, user4 and user3 all appear in both the GUI transcript and the CSV, confirming the logging path works correctly alongside the live GUI.

7. Wireshark Verification

Traffic was captured on port 5000 using the filter tcp.port == 5000 to verify the four required events at the packet level.

Client Connection

930	142.658103249	10.0.0.3	10.0.0.1	TCP	76	36288 → 5000 [SYN] Seq=0 Win=4240 Len=0 MSS=1460 SACK_PERM=1 TSval=2363539945 TSecr=0 WS=512
930	142.658103249	10.0.0.3	10.0.0.1	TCP	76	[TCP Out-of-order] [TCP Port numbers reused] 36288 → 5000 [SYN] Seq=0 Win=4240 Len=0 MSS=1460 SACK_PERM=1 TSval=2363539945 TSecr=0 WS=512
940	142.658103249	10.0.0.3	10.0.0.1	TCP	76	5000 → 36288 [SYN, ACK] Seq=0 Ack=1 Win=4340 Len=0 MSS=1460 SACK_PERM=1 TSval=417822155 TSecr=2363539945 WS=512
941	142.658103249	10.0.0.3	10.0.0.1	TCP	76	[TCP Out-of-order] 5000 → 36288 [SYN, ACK] Seq=0 Ack=1 Win=4340 Len=0 MSS=1460 SACK_PERM=1 TSval=417822155 TSecr=2363539945 WS=512
942	142.658103249	10.0.0.3	10.0.0.1	TCP	68	36288 → 5000 [ACK] Seq=1 Ack=1 Win=4240 Len=0 TSval=2363539946 TSecr=417822155
943	142.658103249	10.0.0.3	10.0.0.1	TCP	68	[TCP Dup ACK 942#1] 36288 → 5000 [ACK] Seq=1 Ack=1 Win=4240 Len=0 TSval=2363539946 TSecr=417822155
944	142.658103249	10.0.0.3	10.0.0.1	TCP	73	36288 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=4240 Len=5 TSval=2363539946 TSecr=417822155
945	142.658103249	10.0.0.3	10.0.0.1	TCP	73	[TCP Retransmission] 36288 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=4240 Len=5 TSval=2363539946 TSecr=417822155
946	142.658103249	10.0.0.1	10.0.0.3	TCP	68	5000 → 36288 [ACK] Seq=1 Ack=6 Win=4350 Len=0 TSval=417822155 TSecr=2363539946
947	142.658103249	10.0.0.1	10.0.0.3	TCP	68	[TCP Dup ACK 946#1] 5000 → 36288 [ACK] Seq=1 Ack=6 Win=4350 Len=0 TSval=417822155 TSecr=2363539946
948	142.661750778	10.0.0.1	10.0.0.3	TCP	132	5000 → 36288 [PSH, ACK] Seq=1 Ack=6 Win=4350 Len=4 TSval=417822155 TSecr=2363539946
949	142.661750778	10.0.0.1	10.0.0.3	TCP	132	[TCP Retransmission] 5000 → 36288 [PSH, ACK] Seq=1 Ack=6 Win=4350 Len=4 TSval=417822155 TSecr=2363539946
950	142.661791804	10.0.0.3	10.0.0.1	TCP	68	36288 → 5000 [ACK] Seq=6 Ack=65 Win=4240 Len=0 TSval=2363539949 TSecr=417822158
951	142.661791804	10.0.0.3	10.0.0.1	TCP	68	[TCP Dup ACK 950#1] 36288 → 5000 [ACK] Seq=6 Ack=65 Win=4240 Len=0 TSval=2363539949 TSecr=417822158

Fig. 7.1 – TCP three-way handshake for a client connecting to the server

The capture shows the SYN from 10.0.0.3 to the server on port 5000, the SYN-ACK reply, and the final ACK completing the handshake, followed by a PSH,ACK carrying the username payload. This confirms the client establishes its TCP connection correctly before any chat data is exchanged.

Broadcast Message

1675	579.989461450	10.0.0.3	10.0.0.1	TCP	95	36288 → 5000 [PSH, ACK] Seq=77 Ack=101 Win=4240 Len=27 TSval=2363977277 TSecr=418219335
1676	579.989461450	10.0.0.3	10.0.0.1	TCP	95	[TCP Retransmission] 36288 → 5000 [PSH, ACK] Seq=77 Ack=101 Win=4240 Len=27 TSval=2363977277 TSecr=418219335
1677	579.989769010	10.0.0.1	10.0.0.3	TCP	68	5000 → 36288 [ACK] Seq=101 Ack=104 Win=4350 Len=0 TSval=418259486 TSecr=2363977277
1678	579.98956805	10.0.0.1	10.0.0.2	TCP	104	5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1679	579.98956805	10.0.0.1	10.0.0.4	TCP	104	5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1680	579.98956805	10.0.0.1	10.0.0.5	TCP	104	5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1681	579.98956805	10.0.0.1	10.0.0.2	TCP	104	[TCP Retransmission] 5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1682	579.98956805	10.0.0.1	10.0.0.1	TCP	68	36082 → 5000 [ACK] Seq=6 Ack=258 Win=4240 Len=0 TSval=193464032 TSecr=252109015
1683	579.98956805	10.0.0.1	10.0.0.4	TCP	104	[TCP Retransmission] 5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1684	579.98956805	10.0.0.1	10.0.0.5	TCP	104	[TCP Retransmission] 5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1685	579.98956805	10.0.0.1	10.0.0.1	TCP	68	[TCP Dup ACK 1677#1] 5000 → 36082 [ACK] Seq=101 Ack=104 Win=4350 Len=0 TSval=418259486 TSecr=2363977277
1686	579.98956805	10.0.0.1	10.0.0.5	TCP	104	[TCP Retransmission] 5000 → 36082 [PSH, ACK] Seq=169 Ack=6 Win=4350 Len=36 TSval=2525180919 TSecr=1793424439
1687	579.98956805	10.0.0.1	10.0.0.1	TCP	68	36082 → 5000 [ACK] Seq=6 Ack=181 Win=4240 Len=0 TSval=193464032 TSecr=252109015
1688	579.98956805	10.0.0.1	10.0.0.2	TCP	68	[TCP Dup ACK 1685#1] 5000 → 36082 [ACK] Seq=101 Ack=104 Win=4350 Len=0 TSval=418259486 TSecr=2363977277
1689	579.98956805	10.0.0.1	10.0.0.4	TCP	68	[TCP Dup ACK 1685#1] 5000 → 36082 [ACK] Seq=101 Ack=104 Win=4350 Len=0 TSval=418259486 TSecr=2363977277
1690	579.98956805	10.0.0.1	10.0.0.5	TCP	68	[TCP Dup ACK 1685#1] 5000 → 36082 [ACK] Seq=101 Ack=104 Win=4350 Len=0 TSval=418259486 TSecr=2363977277

Fig. 7.2 – Server relaying one broadcast message to multiple clients

A single broadcast triggers several near-simultaneous PSH,ACK segments from the server (10.0.0.1) to each connected client (10.0.0.2, 10.0.0.3, 10.0.0.4, 10.0.0.5), each acknowledged separately. This matches the broadcast_system_message() logic, which loops over all online clients and sends the same payload to each socket individually.

Private Message

2139	879.08269804	10.0.0.4	10.0.0.1	TCP	116	59314 → 5000 [PSH, ACK] Seq=28 Ack=315 Win=4240 Len=48 TSval=3554852696 TSecr=3477721659
2140	879.08269804	10.0.0.4	10.0.0.1	TCP	116	[TCP Retransmission] 59314 → 5000 [PSH, ACK] Seq=28 Ack=315 Win=4240 Len=48 TSval=3554852696 TSecr=3477721659
2141	879.08269804	10.0.0.1	10.0.0.4	TCP	68	5000 → 59314 [ACK] Seq=315 Ack=78 Win=4350 Len=0 TSval=3477788889 TSecr=3554852696
2142	879.08269804	10.0.0.1	10.0.0.4	TCP	68	[TCP Dup ACK 2141#1] 5000 → 59314 [ACK] Seq=315 Ack=78 Win=4350 Len=0 TSval=3477788889 TSecr=3554852696
2143	879.08269804	10.0.0.1	10.0.0.3	TCP	127	5000 → 36288 [PSH, ACK] Seq=173 Ack=165 Win=4350 Len=59 TSval=418558579 TSecr=2364209148
2144	879.08269804	10.0.0.1	10.0.0.5	TCP	127	[TCP Retransmission] 5000 → 36288 [PSH, ACK] Seq=173 Ack=165 Win=4350 Len=59 TSval=418558579 TSecr=2364209148
2145	879.08269804	10.0.0.3	10.0.0.1	TCP	68	36288 → 5000 [ACK] Seq=165 Ack=232 Win=4240 Len=0 TSval=2364276370 TSecr=418558579
2146	879.08269804	10.0.0.5	10.0.0.1	TCP	68	[TCP Dup ACK 2145#1] 36288 → 5000 [ACK] Seq=165 Ack=232 Win=4240 Len=0 TSval=2364276370 TSecr=418558579

Fig. 7.3 – Server forwarding a private message to a single client

Unlike the broadcast case, this exchange shows the server sending a PSH,ACK to only one destination IP, acknowledged by that single client. This confirms private messages are delivered point-to-point rather than relayed to every connected socket.

Client Disconnection

2407	10.0.0.1	10.0.0.2	TCP	68	36002 → 5000 [FIN, ACK] Seq=402 Ack=338 Win=42496 Len=0 TSval=1793907340 TSecr=2525615485	
2408	10.0.0.2	10.0.0.1	TCP	68	TCP Out-of-Order 36002 → 5000 [FIN, ACK] Seq=9 Ack=338 Win=42496 Len=0 TSval=1793907340 TSecr=2525615485	
2409	10.0.0.1	10.0.0.2	TCP	68	5000 → 36288 [PSH, ACK] Seq=342 Ack=165 Win=43520 Len=12 TSval=18702195 TSecr=2364411845	
2410	10.0.0.2	10.0.0.1	TCP	68	TCP Retransmission 36288 → 5000 [PSH, ACK] Seq=165 Ack=354 Win=42496 Len=0 TSval=18702195 TSecr=2364411845	
2411	10.0.0.1	10.0.0.2	TCP	68	36288 → 5000 [ACK] Seq=165 Ack=354 Win=42496 Len=0 TSval=18702195 TSecr=2364411845	
2412	10.0.0.2	10.0.0.1	TCP	68	TCP Dup ACK 2411(F) 36288 → 5000 [ACK] Seq=165 Ack=354 Win=42496 Len=0 TSval=18702195 TSecr=2364411845	
2413	10.0.0.1	10.0.0.2	TCP	68	5000 → 35508 [PSH, ACK] Seq=370 Ack=15 Win=43520 Len=12 TSval=1218225342 TSecr=304942338	
2414	10.0.0.2	10.0.0.1	TCP	68	TCP Out-of-Order 35508 → 5000 [PSH, ACK] Seq=15 Ack=382 Win=42496 Len=0 TSval=304950479 TSecr=1218225342	
2415	10.0.0.1	10.0.0.2	TCP	68	35508 → 5000 [ACK] Seq=15 Ack=382 Win=42496 Len=0 TSval=304950479 TSecr=1218225342	
2416	10.0.0.2	10.0.0.1	TCP	68	TCP Dup ACK 2415(F) 35508 → 5000 [ACK] Seq=15 Ack=382 Win=42496 Len=0 TSval=304950479 TSecr=1218225342	
2417	10.0.0.1	10.0.0.2	TCP	68	5000 → 36002 [FIN, ACK] Seq=63 Ack=339 Win=42496 Len=0 TSval=2525623631 TSecr=1793907340	
2418	10.0.0.2	10.0.0.1	TCP	68	TCP Out-of-Order 36002 → 5000 [FIN, ACK] Seq=338 Ack=63 Win=43520 Len=0 TSval=1793907340 TSecr=2525623631	
2419	10.0.0.1	10.0.0.2	TCP	68	36002 → 5000 [ACK] Seq=63 Ack=339 Win=42496 Len=0 TSval=2525623631 TSecr=1793907340	
2420	10.0.0.2	10.0.0.1	TCP	68	TCP Dup ACK 2419(F) 36002 → 5000 [ACK] Seq=63 Ack=339 Win=42496 Len=0 TSval=1793907340 TSecr=2525623631	

Fig. 7.4 – FIN/ACK exchange as a client disconnects

The capture shows FIN,ACK segments exchanged between the client and server, along with retransmissions of the final LEAVE notification before the connection fully closes. This corresponds to the client's `disconnect()` method closing its socket and the server detecting the closed connection in its receive loop.

8. Reflection Answers

1. Why should networking logic and GUI code be separated?

Keeping them separate meant I could reuse my entire Assignment 5 server and socket logic without touching it — only `client_gui.py` changed. It also keeps responsibilities clear: the socket layer only cares about sending and receiving bytes, while the GUI only cares about displaying them. If they were mixed together, a change to the interface could easily break the protocol handling, and vice versa.

2. Why is a background thread required?

Tkinter's main loop has to stay free to redraw the window and respond to clicks and key presses. `socket.recv()` blocks until data arrives, so if it ran on the main thread the whole GUI would freeze every time it waited for a message. Running `receive_messages()` in a separate daemon thread lets the client listen for incoming data continuously while the user can still type and send messages at any time.

3. Which components from Assignment 5 were reused?

The entire server — socket setup, per-client worker threads, broadcast and private messaging, the clients state dictionary, and CSV-based chat history logging — was reused with essentially no changes. On the client side, the underlying `connect/send/recv` calls and the `/msg` command format were reused; only the terminal `input()/print()` calls were replaced with GUI widgets and callbacks.

4. How did the GUI improve usability?

In the Assignment 5 terminal client, typing a private message meant remembering and typing the exact `/msg <username> <message>` syntax, and there was no visual list of who was online. In the GUI, online users are shown in a list, and selecting a name automatically formats a private message for that user. The scrollable chat window also makes it much easier to follow a fast-moving conversation than a plain terminal feed.

5. Suggested future enhancement:

Adding a password field with basic authentication would be a natural next step, since the current login only checks that a username is non-empty and does not verify identity at all.

9. Conclusion

This assignment successfully converted the Assignment 5 terminal-based chat client into a functional Tkinter GUI application while reusing the server and networking logic without modification. Testing across four

simultaneous Mininet clients confirmed that login, broadcast messaging, private messaging, join/leave notifications, and disconnection all work correctly, and Wireshark captures verified that each of these events produces the expected TCP behaviour at the packet level. The exercise reinforced why separating networking code from presentation code matters, and why blocking socket calls must be kept off the GUI thread.